Applied Machine Learning

Neural Networks: Practical Aspects

UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

richard.johansson@cse.gu.se

overview

overview of neural network libraries

tricks of the trade

training efficiency of NNs

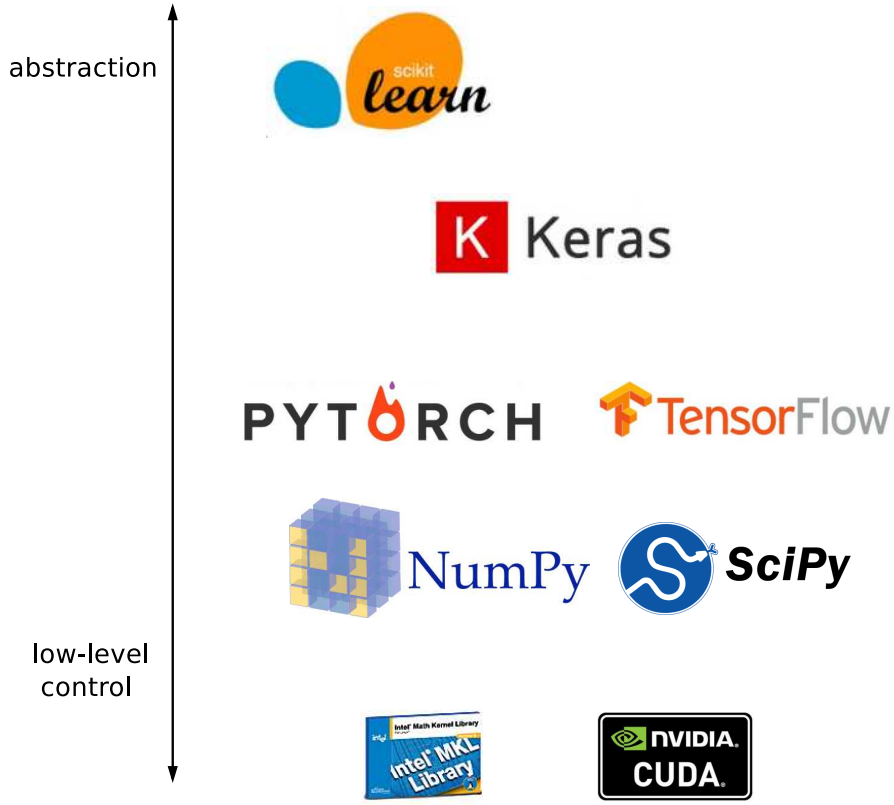
- our previous classifiers took seconds or minutes to train
- NNs tend to take minutes, hours, days, weeks . . .
  - depending on the complexity of the network and the amount of training data
- NNs use a lot of linear algebra (matrix multiplications) so it can be useful to work to speed up the math
  - parallelize as much as possible
  - use optimized math libraries
  - use a GPU
  - no need to implement backprop
  - in short: you're better of using a specialized NN library

neural network software: Python

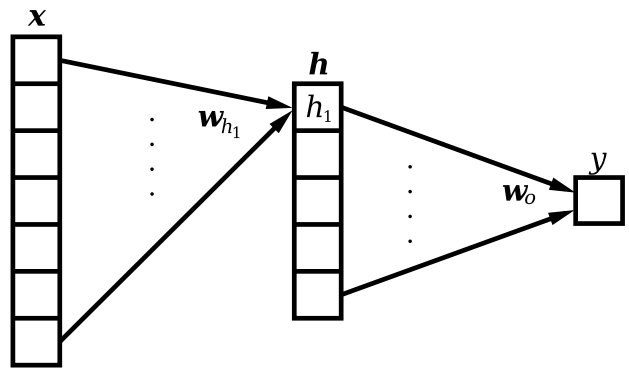
- scikit-learn currently has quite limited support for NNs
- the main NN software in the Python world used to be **Theano**
  - developed by Yoshua Bengio's group in Montréal
  - `http://deeplearning.net/software/theano`
- the major players have released their own libraries in the last few years:
  - Google: TensorFlow
  - Facebook: PyTorch
  - Microsoft: CNTK
- these toolkits provide "building blocks" such as layers, activations, losses, regularizers, optimizers, ...

neural network software: Python (2)

- TensorFlow etc. do a lot of useful math stuff, and integrate nicely with the GPU, but they can be a bit low-level
- so there are libraries that create a more high-level interface, a bit similar to scikit-learn
  - **Keras**
  - required for Assignment 5



coding example with Keras




```
keras_model = Sequential()

n_hidden = 3
keras_model.add(Dense(input_dim=X.shape[1],
                      output_dim=n_hidden))
keras_model.add(Activation("sigmoid"))

keras_model.add(Dense(input_dim=n_hidden,
                      output_dim=1))
keras_model.add(Activation("sigmoid"))

keras_model.compile(loss='binary_crossentropy',
                    optimizer='sgd')

keras_model.fit(X, Y)
```

overview

overview of neural network libraries

tricks of the trade

feature preprocessing

- NNs are very sensitive to feature preprocessing
- depending on the scale of a feature, some units may be "saturated" and be difficult to optimize
- if features have different magnitudes, they may be affected differently by regularizers
- it is good practice to scale numerical features
  - for instance scikit-learn's `StandardScaler` or `MinMaxScaler`

processing one instance at a time is inefficient

- as we have discussed, it is more efficient to carry out large-scale linear algebra operations
- but SGD works incrementally, one instance at a time!
- **minibatch** gradient descent: small subsets instead of single instances
- in Keras:

```
model.fit(X, Y, batch_size=400)
```

minibatch gradient descent: pseudocode

initialize $\boldsymbol{w}$

repeat ...

  select a small batch (subset) $\boldsymbol{X}_b$, Y_b from the $\boldsymbol{X}$, Y

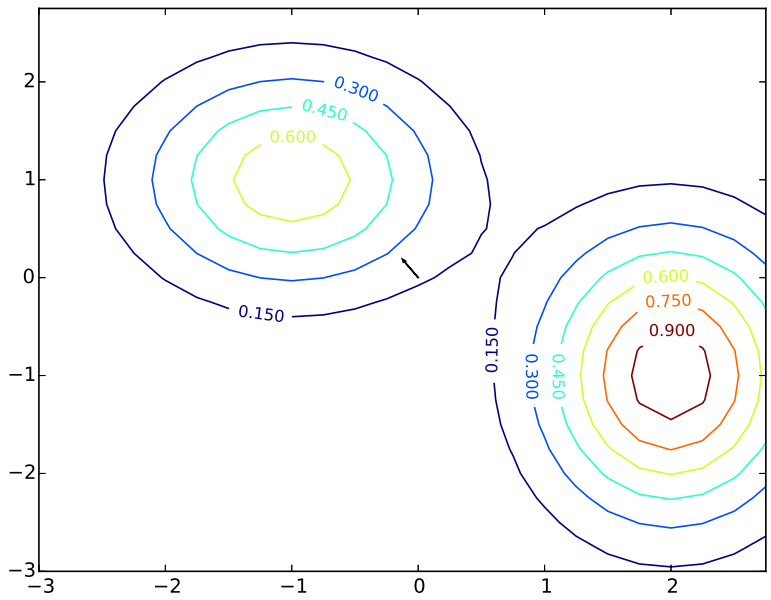
  compute gradient ∇f_b of the loss for current batch $\boldsymbol{X}_b$, Y_b

  $\boldsymbol{w} = \boldsymbol{w} - \eta \cdot \nabla f_b(\boldsymbol{w})$

return $\boldsymbol{w}$

optimizing NNs

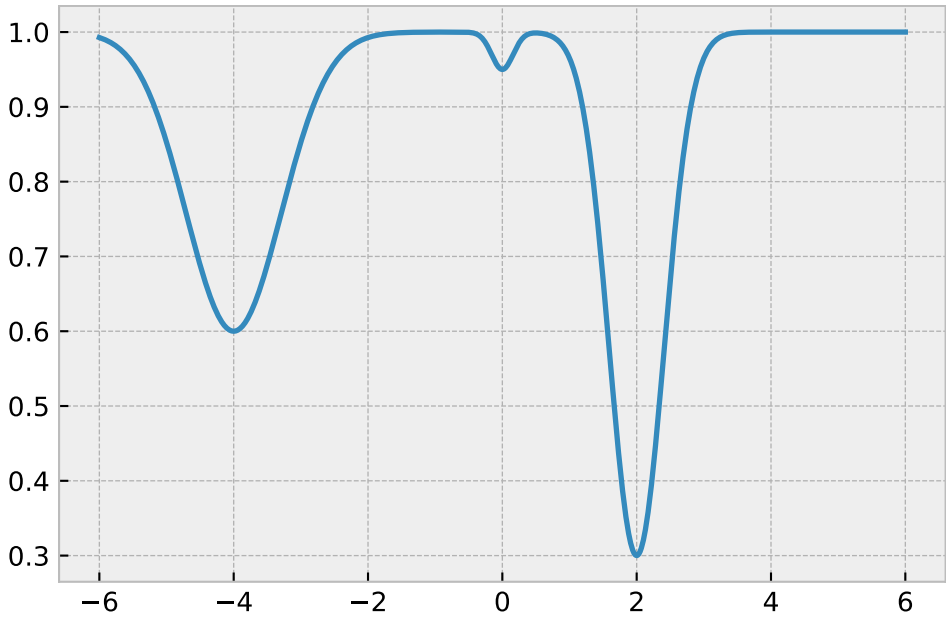
- unlike the linear classifiers we studied previously, NNs have non-convex objective functions with a lot of local minima
- so the end result depends on initialization



- when working with NNs, it is good practice to re-run with different initializations (e.g. different random seeds)

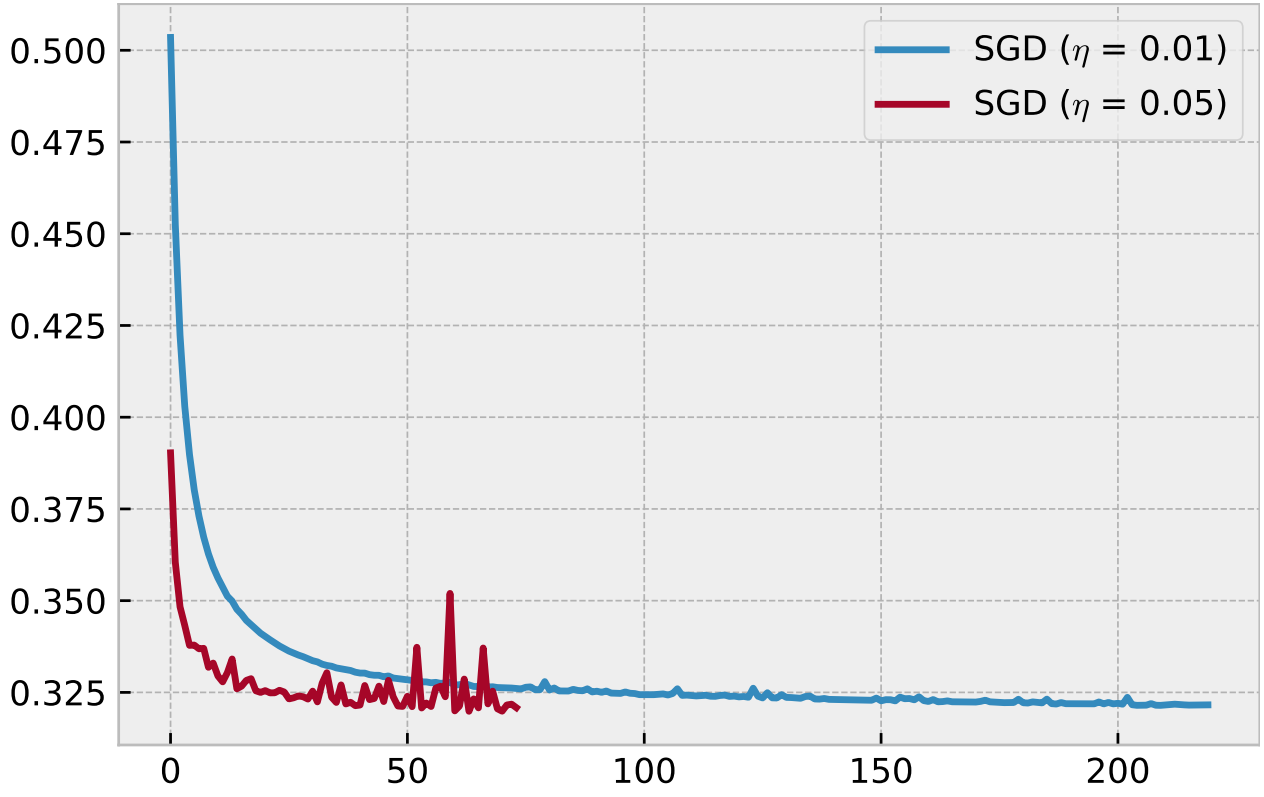
plateaus in the objective

- NN objectives tend to have plateaus:



- this irregular shape makes it hard to set the learning rate

example: two different learning rates



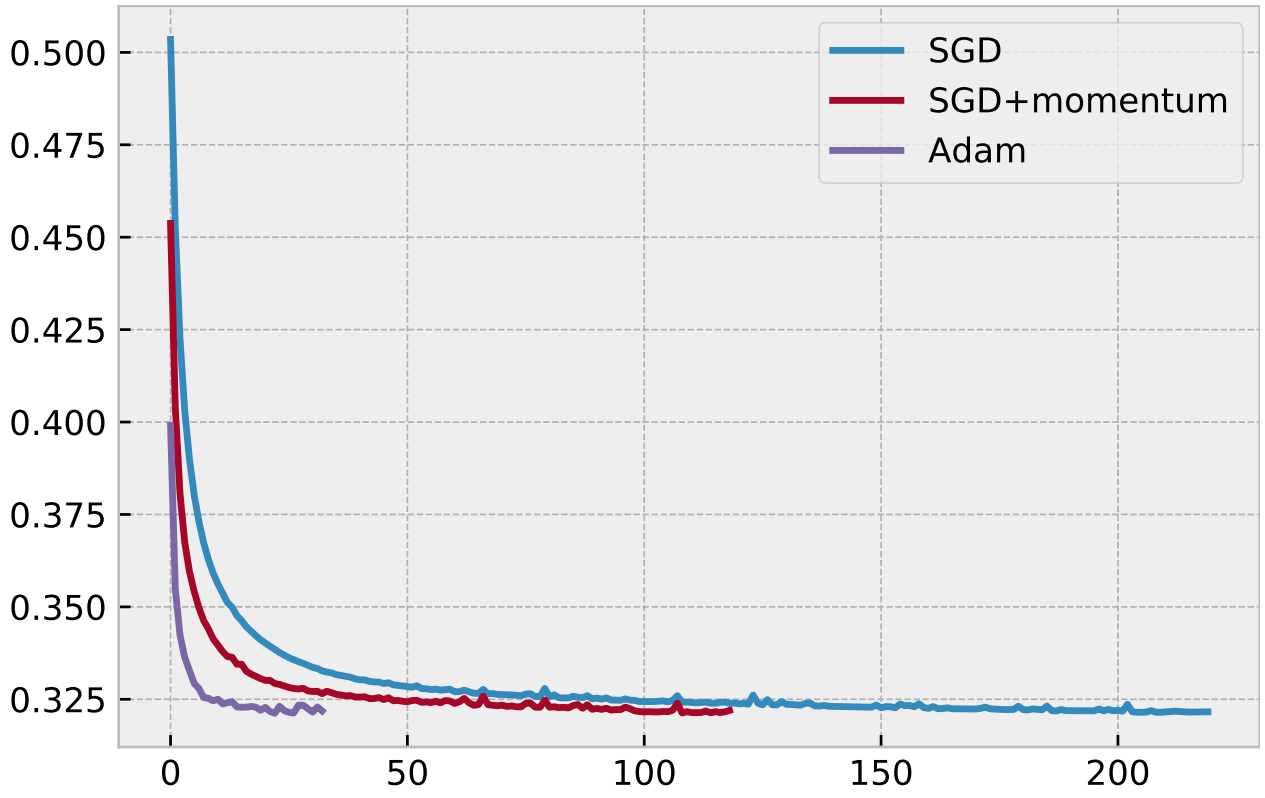
adaptive gradient updates

- **adaptive** gradient descent methods control η to accelerate and slow down when necessary
- popular adaptive methods: **Adam**, **Adagrad**, **RMSProp**, ...
- in Keras:

```
model.compile(..., optimizer='adam', ...)
```

- see Sebastian Ruder's report An overview of gradient descent optimization algorithms for an overview

example: comparison of optimizers



avoiding overfitting

- we've already seen how to apply a **regularizer** for logistic regression and SVC

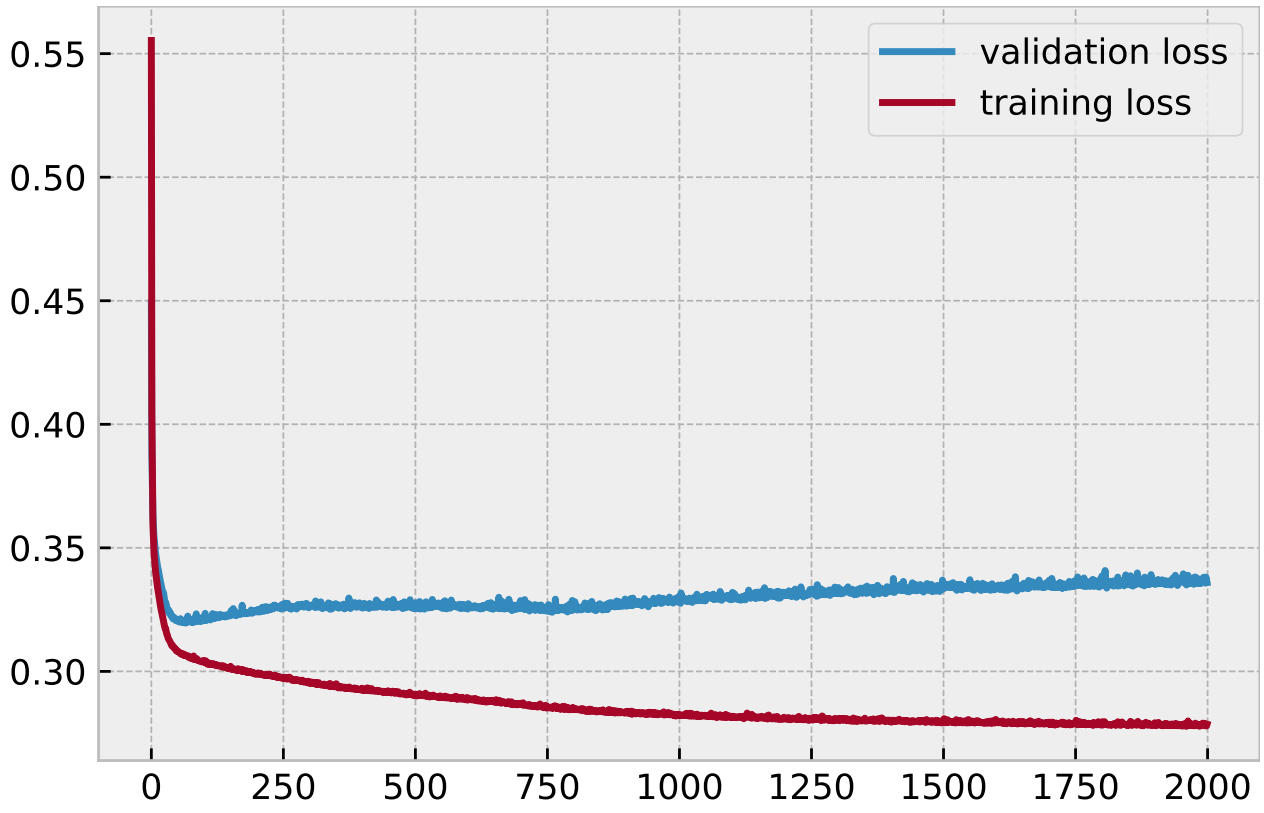
$$\sum \text{Loss}(x_i, y_i, \boldsymbol{w}) + R(\boldsymbol{w})$$

  where $R(\boldsymbol{w})$ can be $\|\boldsymbol{w}\|^2$, for instance
- in Keras:

```
from keras import regularizers
regularizer = regularizers.l2(0.001)
```

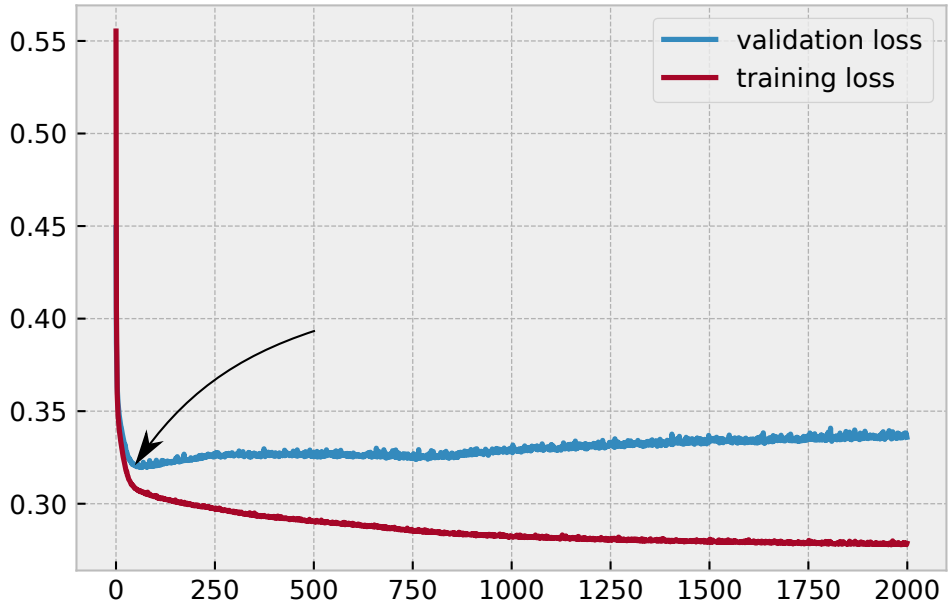
- in the neural network world, there are a few other methods that are also popular, such as **early stopping** and **dropout**

overfitting: example



early stopping

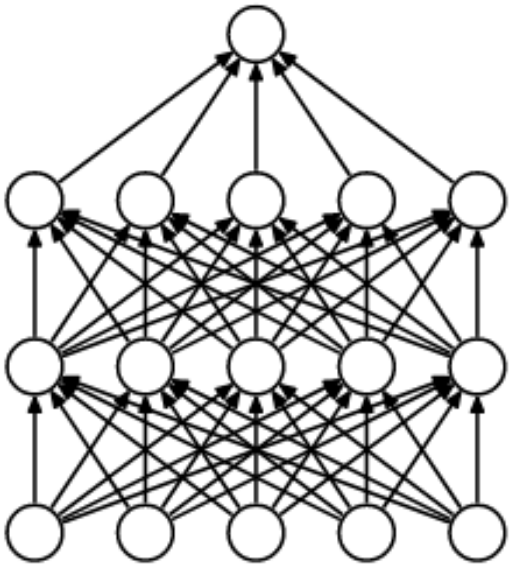
- reserve a held-out development set (validation set)
- terminate training when there is no improvement on the held-out data



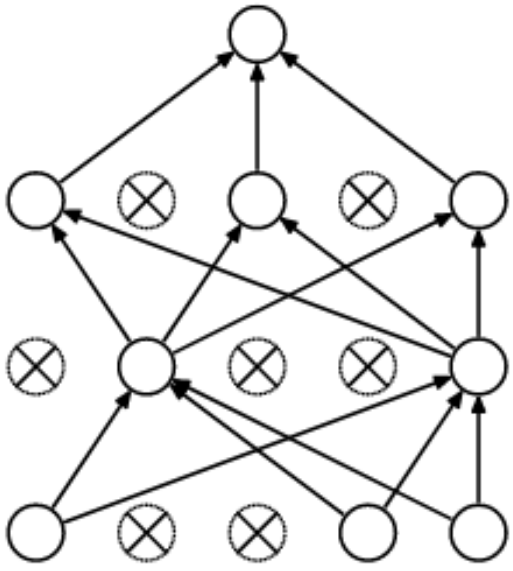
early stopping: Keras

```
from keras import callbacks

cb = callbacks.EarlyStopping(monitor='val_loss', min_delta=0,
                             patience=10, verbose=0, mode='auto')

...

model.fit(X, Y, epochs=1000, batch_size=400, verbose=2,
          validation_split=0.1, callbacks=[cb])
```

dropout



(a) Standard Neural Net

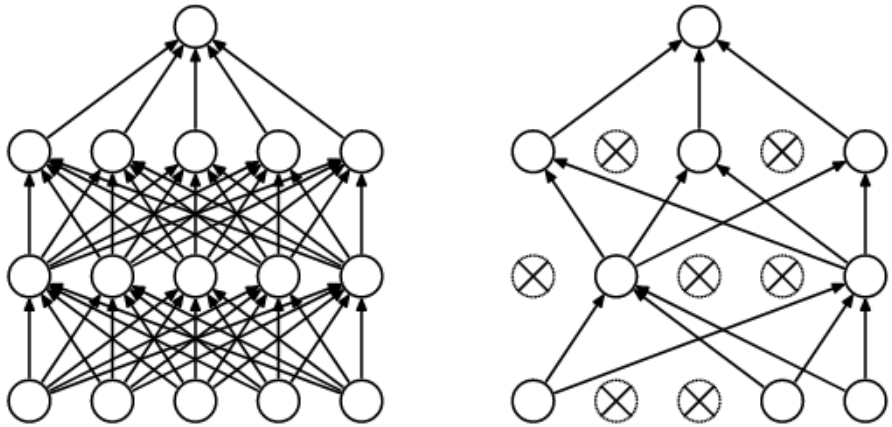


(b) After applying dropout.

[source]

dropout in neural networks

- **dropout** is often used to reduce the risk of overfitting in neural networks



(a) Standard Neural Net (b) After applying dropout.

[source]

- why does it work?
- Srivastava et al. (2014) motivate dropout in terms of ensembles
  - if there are N connections in the model, we can see it as an ensemble of 2^N different models (subsets of connections)

dropout: Keras

```
from keras.layers import Dropout

model = Sequential()
model.add(Dense(... something ...))
model.add(Dropout(0.1))
model.add(Dense(... something ...))

model.compile(...)
modle.fit(...)
```

noising, data augmentation, . . .

- to reduce overfitting, we can add some random **noise** to training instances
  - for linear regression: Gaussian noise $\Leftrightarrow$ L_2 regularization
- more generally, we can apply **data augmentation** to expose our model to more variation during training
  - for instance, blurring or rotating images

summary

- for NN development, we will use libraries such as Keras, PyTorch or TensorFlow
- practical things to keep in mind:
  - **scaling** numerical features
  - **minibatching**: processing instances in parallel
  - **optimization**: SGD and its adaptive variants
  - regularization including **early stopping**, **dropout**, and **noising** to avoid overfitting